

PLAYWRIGHT AUTOMATION TESTING PROJECT REPORT

Submitted By

Anisha Kumari

B.Tech – Computer Science & Information Technology

Submission Date

04 June 2026 – 05 June 2026

CERTIFICATE

This is to certify that i have successfully completed the project titled “E-Commerce Website Automation Testing Using Playwright” as part of the training requirements. The project demonstrates implementation of automation testing concepts using Playwright Framework.

ACKNOWLEDGEMENT

I express my sincere gratitude to my mentors, faculty members, and Wipro training team for their guidance and support throughout the development of this automation testing project.

ABSTRACT

This project focuses on automating the testing of an E-Commerce website using Playwright and JavaScript.

The framework follows the Page Object Model (POM) design pattern and includes UI Automation Testing, Cross Browser Testing, API Testing, Assertions, Parameterized Testing, Allure Reporting, and GitHub Actions CI/CD.

The project automates major business modules such as Authentication, Product Management, Cart Management, Checkout Process, Shipping Flow, Customer Support, User Profile Management, and API Validation.

TABLE OF CONTENTS

1. Introduction
 2. Project Overview
 3. Objectives
 4. Technologies Used
 5. Framework Architecture
 6. Modules Covered
 7. Test Execution Summary
 8. Cross Browser Testing
 9. API Testing
 10. Reports and Results
 11. GitHub Deployment
 12. Conclusion
-

1. INTRODUCTION

Software Testing ensures application quality, reliability, and performance. Automation Testing reduces manual effort and increases execution speed.

Playwright is a modern automation testing framework that supports Chromium, Firefox, and WebKit browsers with powerful automation capabilities.

2. PROJECT OVERVIEW

Project Name: E-Commerce Website Automation Testing

Application Under Test: <https://automationexercise.com>

Automation Tool: Playwright

Language: JavaScript

Framework Design: Page Object Model (POM)

Reporting: HTML Report and Allure Report

Version Control: GitHub

CI/CD: GitHub Actions

3. OBJECTIVES

- Automate critical business flows
 - Validate application functionality
 - Execute tests across browsers
 - Generate execution reports
 - Improve testing efficiency
 - Demonstrate framework design principles
-

4. TECHNOLOGIES USED

Technology	Purpose
Playwright	UI & API Automation
JavaScript	Programming Language
Node.js	Runtime Environment
GitHub	Version Control
GitHub Actions	CI/CD
Allure Report	Reporting
VS Code	Development Environment

5. FRAMEWORK ARCHITECTURE

Framework Pattern: Page Object Model (POM)

Project Structure:

pages/ tests/ auth/ product/ cart/ checkout/ shipping/ support/ profile/ api/

playwright.config.js

package.json

.github/workflows/playwright.yml

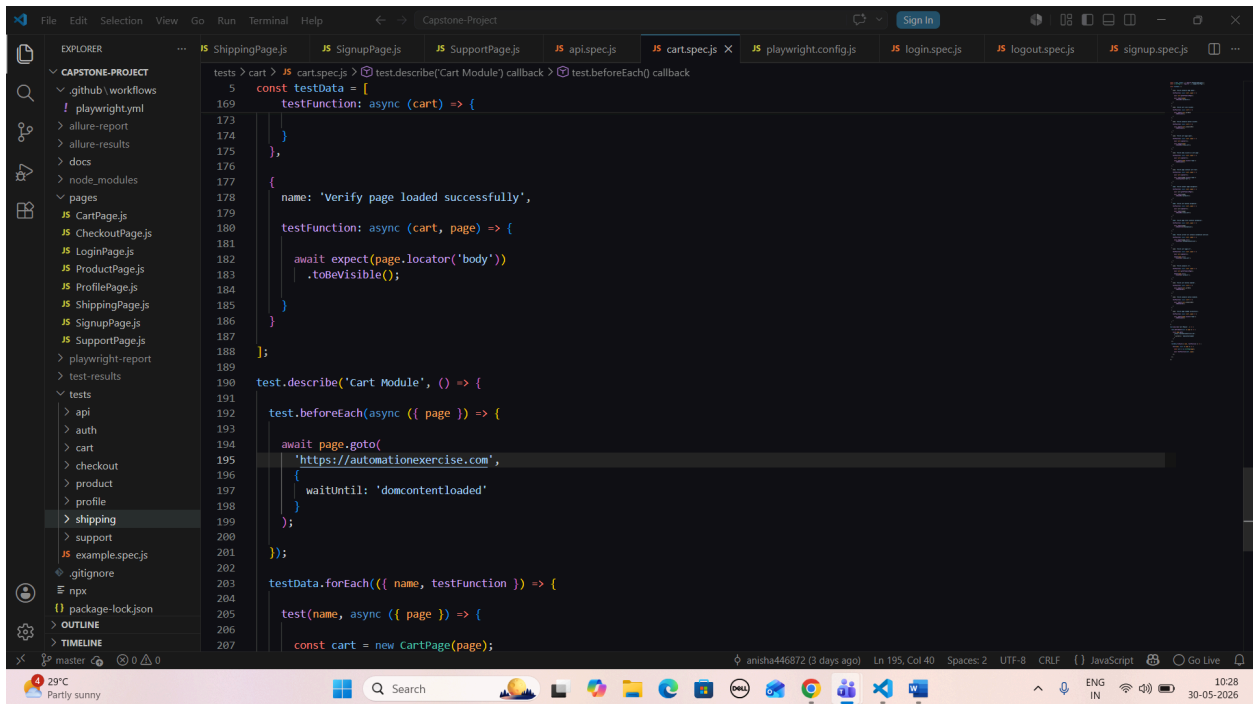


Figure 5.1 – Project Folder Structure

6. MODULES COVERED

6.1 Authentication Module

Purpose: Validate user authentication functionality.

Test Cases Covered:

- Login page opens
- Email field visible
- Password field visible
- Login button visible
- Login button enabled
- Invalid login validation

- Signup page opens
- Signup form validation
- Logout functionality
- Forgot password validation

The image shows a web browser window at the top displaying a list of test results for 'Logout Tests'. The tests are categorized by browser (chromium, firefox, webkit) and their durations. Below the browser window, a VS Code editor is open, showing the source code for 'LoginPage.js' and a terminal window displaying a failed test report. The test report indicates a failure in the 'Logout Tests > Verify user can logout successfully' test, with a screenshot and video attachment. The error context shows the test failed at line 28 of 'LoginPage.js'.

Test Name	Browser	Duration
Logout Tests > test name	chromium	3ms
Slow logout test example	chromium	5.7s
Verify logout button is visible after successful login	chromium	6.2s
Logout Tests > Verify user can logout successfully	firefox	7.0s
Logout Tests > test name	firefox	3ms
Slow logout test example	firefox	13.0s
Verify logout button is visible after successful login	firefox	5.5s
Logout Tests > Verify user can logout successfully	webkit	8.9s
Logout Tests > test name	webkit	8ms
Slow logout test example	webkit	6.9s
Verify logout button is visible after successful login	webkit	10.8s

```

// Logout action
async logout() {
  await this.logoutBtn.click();
}

```

attachment #1: screenshot (image/png)
test-results\auth-logout-Logout-Tests-Verify-user-can-logout-successfully-chromium\test-failed-1.png

attachment #2: video (video/webm)
test-results\auth-logout-Logout-Tests-Verify-user-can-logout-successfully-chromium\video.webm

Error Context: test-results\auth-logout-Logout-Tests-Verify-user-can-logout-successfully-chromium\error-context.md

1 failed
[chromium] > tests\auth\logout.spec.js:28:5 > Logout Tests > Verify user can logout successfully
3 skipped
41 passed (5.6m)

Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.

Concepts Used:

- POM

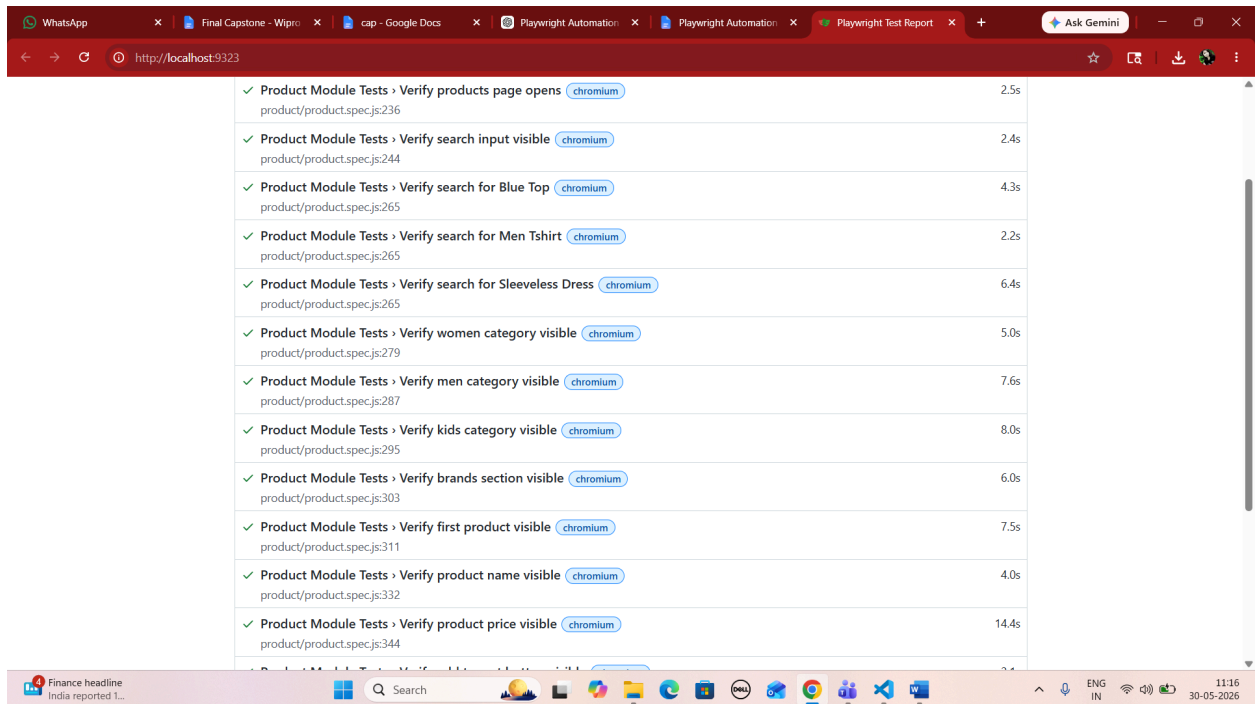
- Assertions
- Hooks
- Cross Browser Testing

6.2 Product Module

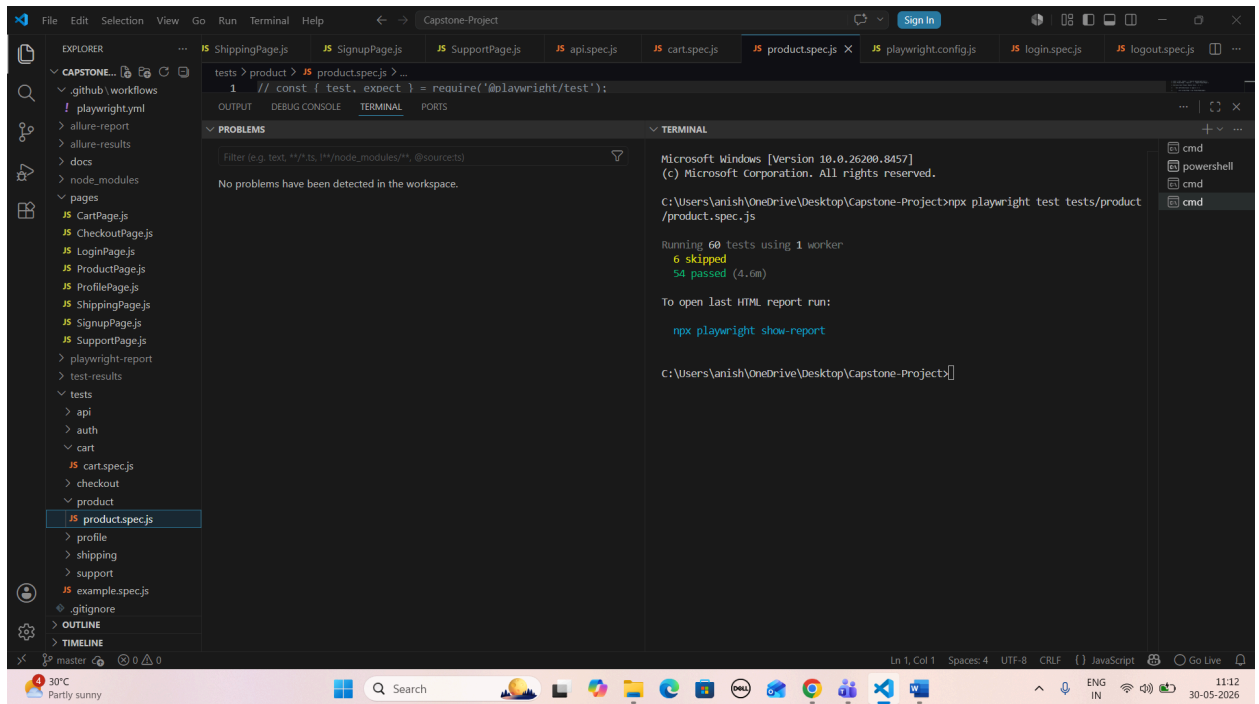
Purpose: Validate product catalog and navigation.

Test Cases Covered:

- Products page opens
- Search product
- Category visibility
- Brand visibility
- Product details validation
- Product name validation
- Product price validation
- Add to cart validation
- Product navigation validation



✓ Product Module Tests › Verify products page opens	product/product.spec.js:236	chromium	2.5s
✓ Product Module Tests › Verify search input visible	product/product.spec.js:244	chromium	2.4s
✓ Product Module Tests › Verify search for Blue Top	product/product.spec.js:265	chromium	4.3s
✓ Product Module Tests › Verify search for Men Tshirt	product/product.spec.js:265	chromium	2.2s
✓ Product Module Tests › Verify search for Sleeveless Dress	product/product.spec.js:265	chromium	6.4s
✓ Product Module Tests › Verify women category visible	product/product.spec.js:279	chromium	5.0s
✓ Product Module Tests › Verify men category visible	product/product.spec.js:287	chromium	7.6s
✓ Product Module Tests › Verify kids category visible	product/product.spec.js:295	chromium	8.0s
✓ Product Module Tests › Verify brands section visible	product/product.spec.js:303	chromium	6.0s
✓ Product Module Tests › Verify first product visible	product/product.spec.js:311	chromium	7.5s
✓ Product Module Tests › Verify product name visible	product/product.spec.js:332	chromium	4.0s
✓ Product Module Tests › Verify product price visible	product/product.spec.js:344	chromium	14.4s



Concepts Used:

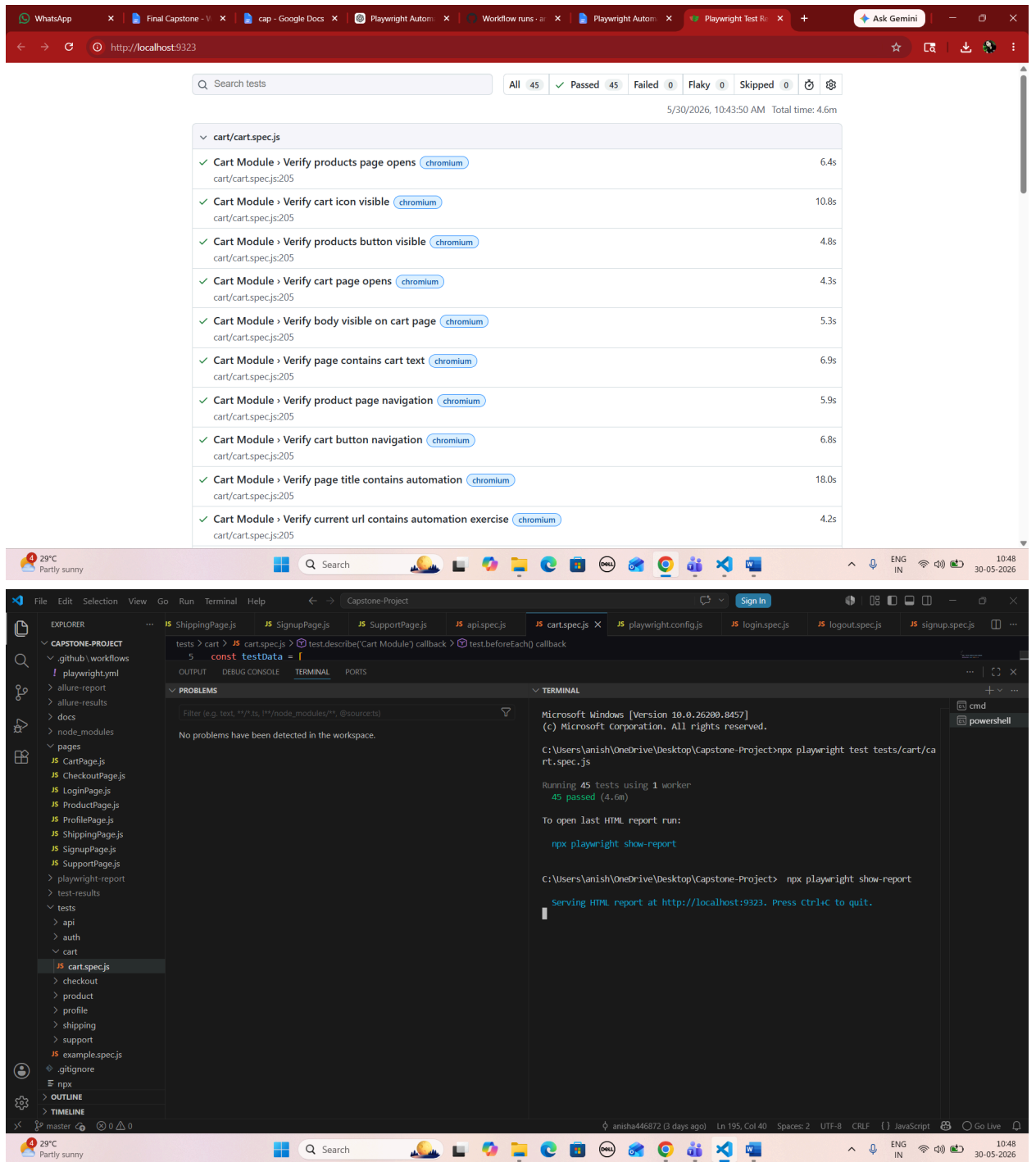
- POM
- Assertions
- Parameterized Testing

6.3 Cart Module

Purpose: Validate cart functionality.

Test Cases Covered:

- Cart page opens
- Cart button visible
- Cart button enabled
- Products button visible
- Cart URL validation
- Page title validation
- Page load validation
- Footer validation



Concepts Used:

- POM
- Assertions
- Navigation Testing

6.4 Checkout Module

Purpose: Validate checkout workflow.

Test Cases Covered:

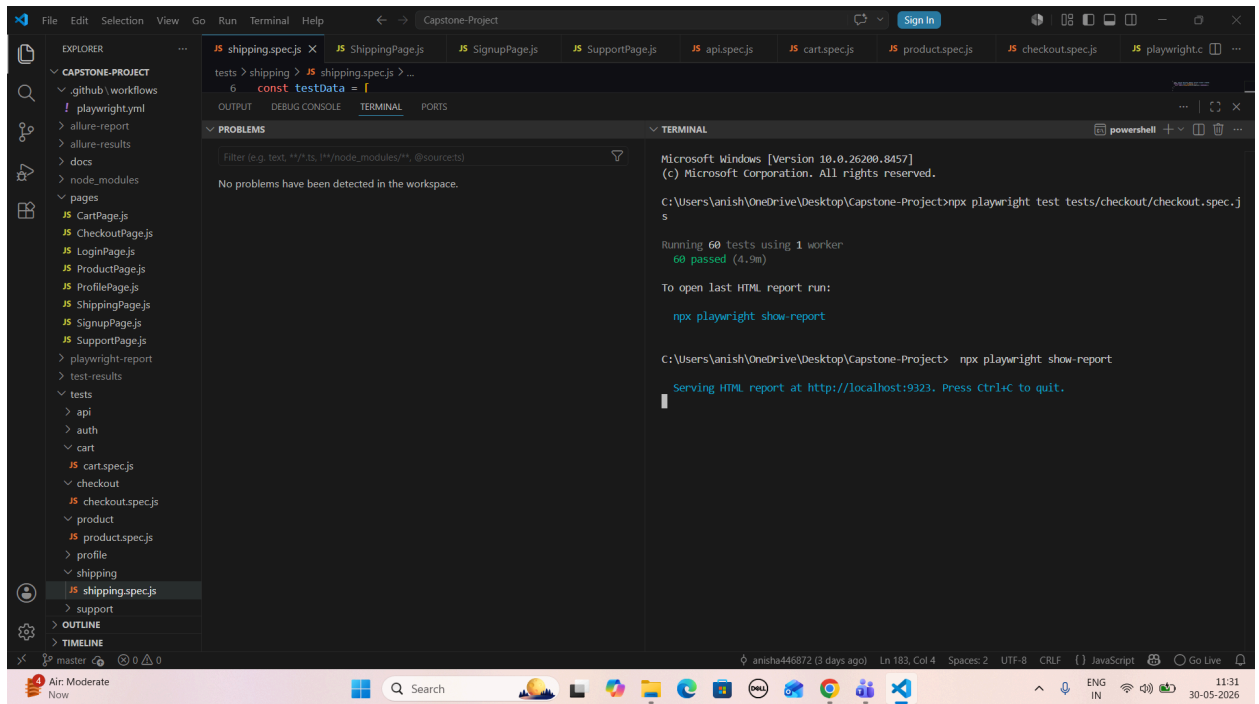
- Checkout button validation
- Cart navigation
- Products navigation
- Login navigation
- Contact page navigation
- Footer validation
- Subscription validation
- Checkout page visibility

The screenshot displays a web browser window with multiple tabs open, including WhatsApp, Final Capstone, Google Docs, and several Playwright test runner tabs. The active tab shows the Playwright test results for a file named `checkout/checkout.spec.js`. The browser's address bar shows `http://localhost:9323`. The test results interface includes a search bar, a summary bar, and a detailed list of test cases.

Summary: 5/30/2026, 11:18:17 AM Total time: 4.9m

Test Case	Duration	Status
✓ Checkout And Order Module › Verify products page opens	3.2s	Passed
✓ Checkout And Order Module › Verify cart page opens	4.0s	Passed
✓ Checkout And Order Module › Verify body visible on cart page	3.8s	Passed
✓ Checkout And Order Module › Verify cart page contains cart text	8.7s	Passed
✓ Checkout And Order Module › Verify products page body visible	2.2s	Passed
✓ Checkout And Order Module › Verify products button visible	2.4s	Passed
✓ Checkout And Order Module › Verify cart button visible	1.8s	Passed
✓ Checkout And Order Module › Verify products button enabled	2.4s	Passed
✓ Checkout And Order Module › Verify cart button enabled	4.5s	Passed
✓ Checkout And Order Module › Verify home button visible	5.3s	Passed

The bottom of the screenshot shows a Windows taskbar with a search bar, various application icons, and system status information including a temperature of 30°C, weather of 'Partly sunny', and the date/time 11:23 on 30-05-2026.



Concepts Used:

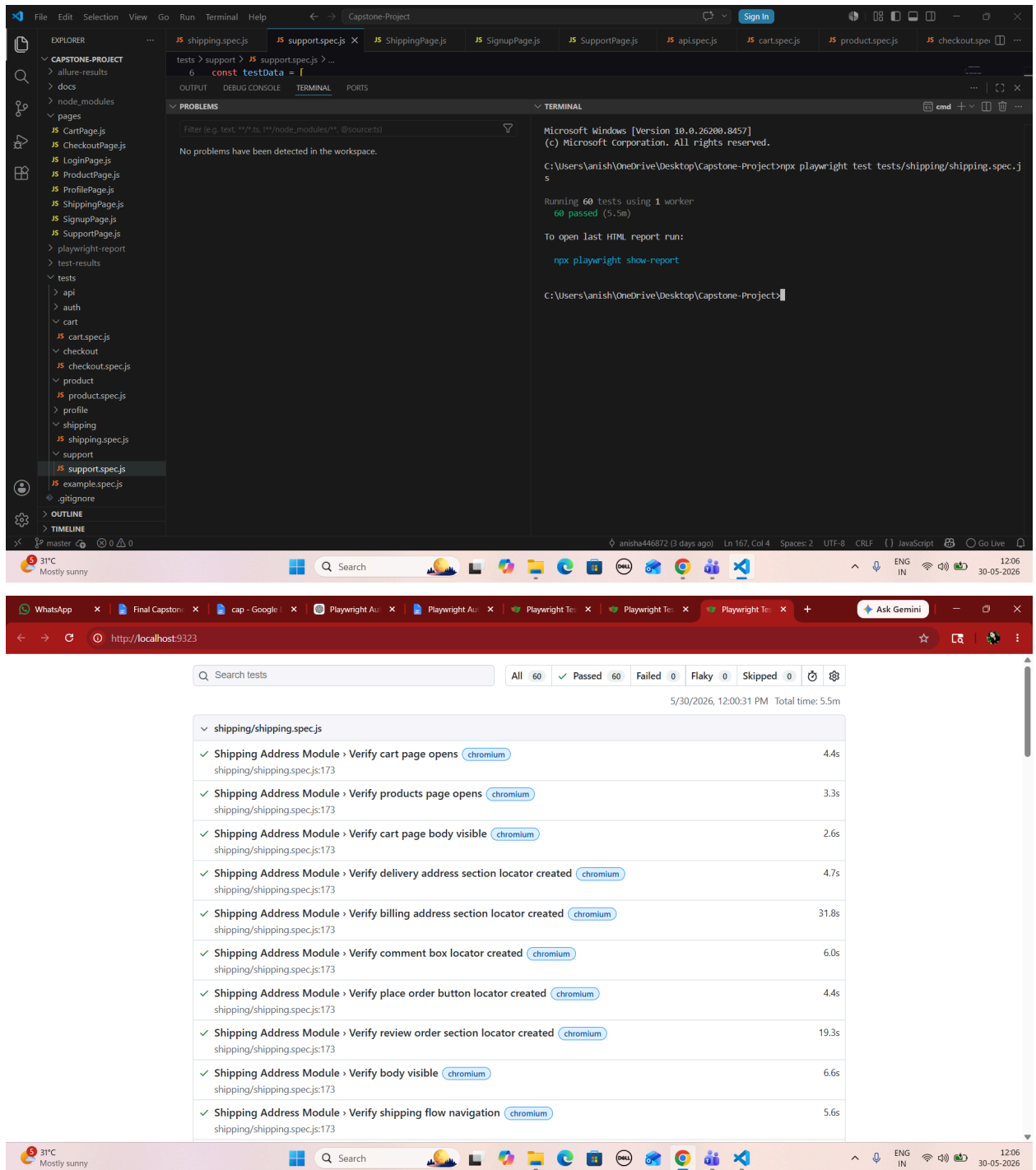
- POM
- Assertions
- Navigation Testing

6.5 Shipping Module

Purpose: Validate shipping workflow and address handling.

Test Cases Covered:

- Cart page opens
- Products page opens
- Delivery address locator
- Billing address locator
- Comment box locator
- Place order locator
- Review order locator
- Footer validation
- Subscription validation
- Shipping navigation validation



Concepts Used:

- POM
- Assertions
- UI Validation

6.6 Customer Support Module

Purpose: Validate customer support functionality.

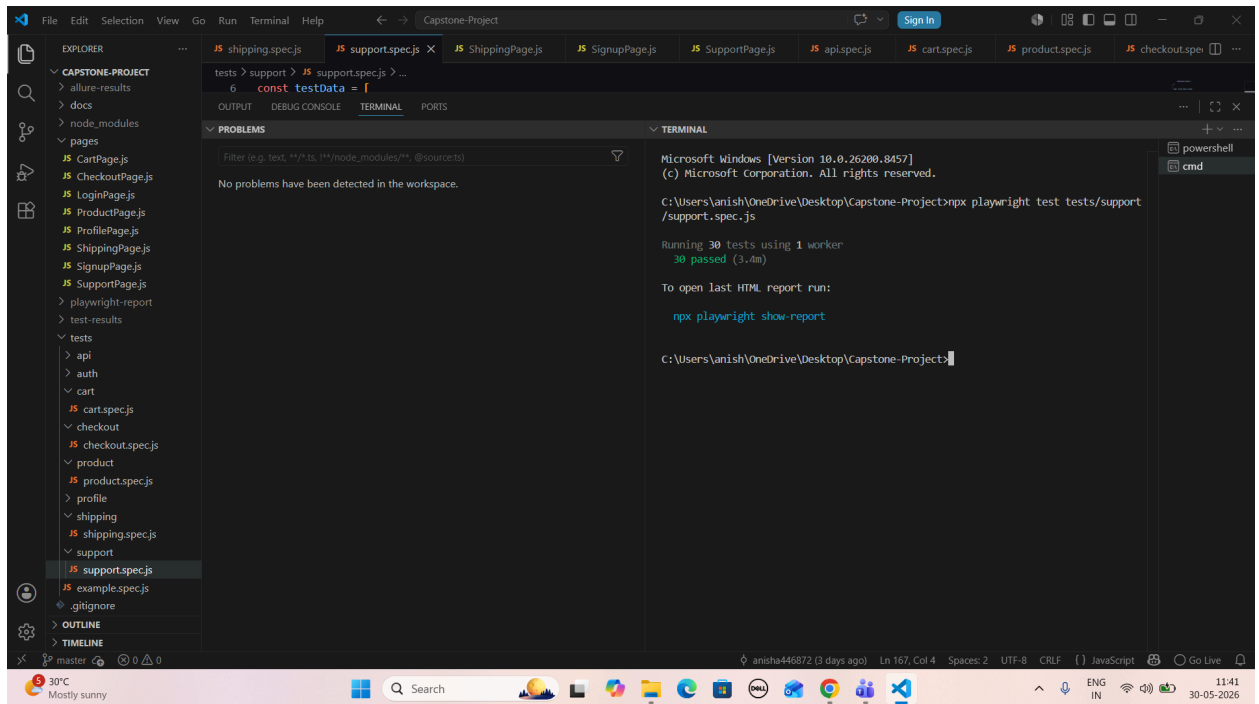
Test Cases Covered:

- Contact page opens
- Name field validation
- Email field validation
- Subject field validation
- Message field validation
- Submit button validation
- Form visibility validation

The screenshot displays a web browser window with a Playwright Test Report. The browser's address bar shows 'http://localhost:9323'. The report is titled 'support/support.spec.js' and shows a summary of 10 tests, all of which passed. The tests are listed in a table with columns for the test name, the browser used (Chromium), and the duration. The tests are:

Test Name	Browser	Duration
Customer Support Module › Verify contact page opens	chromium	3.4s
Customer Support Module › Verify name field visible	chromium	3.5s
Customer Support Module › Verify email field visible	chromium	4.1s
Customer Support Module › Verify subject field visible	chromium	5.2s
Customer Support Module › Verify message field visible	chromium	5.0s
Customer Support Module › Verify submit button visible	chromium	5.1s
Customer Support Module › Verify form fields enabled	chromium	4.9s
Customer Support Module › Verify contact form accepts data	chromium	8.3s
Customer Support Module › Verify footer visible	chromium	6.2s
Customer Support Module › Verify page loaded successfully	chromium	4.7s

The report also shows a search bar, a summary of results (All 30, Passed 30, Failed 0, Flaky 0, Skipped 0), and a timestamp of 5/30/2026, 11:27:03 AM. The total time for the tests is 4.0m.



Concepts Used:

- Form Validation
- Assertions
- POM

6.7 User Profile Module

Purpose: Validate user profile related functionality.

Test Cases Covered:

- Login page validation
- Email field validation
- Password field validation
- Login button validation
- Form enabled validation
- Logout locator validation

The image shows a development environment with VS Code and a web browser. In VS Code, the file explorer on the left shows a project structure for 'CAPSTONE-PROJECT' with various test files. The terminal on the right shows the command `npm playwright test tests/profile/profile.spec.js` being executed, resulting in 30 tests passing. Below the VS Code window, a web browser displays the Playwright Test Report at `http://localhost:9323`. The report shows a list of tests for the 'User Profile Module' that all passed successfully.

VS Code Terminal Output:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\anish\OneDrive\Desktop\Capstone-Project>npx playwright test tests/profile
/profile.spec.js

Running 30 tests using 1 worker
30 passed (2.8m)

To open last HTML report run:

npx playwright show-report

C:\Users\anish\OneDrive\Desktop\Capstone-Project>
```

Playwright Test Report:

Test Name	Duration	Status
User Profile Module › Verify login page opens	2.7s	Passed
User Profile Module › Verify email field visible	3.0s	Passed
User Profile Module › Verify password field visible	3.6s	Passed
User Profile Module › Verify login button visible	3.5s	Passed
User Profile Module › Verify email field enabled	4.3s	Passed
User Profile Module › Verify password field enabled	6.8s	Passed
User Profile Module › Verify login button enabled	4.5s	Passed
User Profile Module › Verify login form accepts data	5.1s	Passed
User Profile Module › Verify logout button locator exists	3.9s	Passed
User Profile Module › Verify page loaded successfully	5.1s	Passed

Concepts Used:

- Assertions
- UI Validation
- POM

6.8 API Testing Module

Purpose: Validate REST API functionality.

Test Cases Covered:

- GET Request
- POST Request
- PUT Request
- DELETE Request
- Status Code Validation
- Header Validation
- JSON Validation
- Response Body Validation
- Response Time Validation

The top screenshot shows a web browser at <http://localhost:9323> displaying Playwright test results. The results table shows 60 tests passed in 20.5s. The tests are categorized by module and test name.

Module	Test Name	Duration
api/api.spec.js	API Testing Module > GET Posts API	442ms
api/api.spec.js:6	API Testing Module > GET Single Post API	1.2s
api/api.spec.js:17	API Testing Module > Validate Post ID	124ms
api/api.spec.js:28	API Testing Module > Validate User ID Exists	124ms
api/api.spec.js:41	API Testing Module > POST Create Post API	787ms
api/api.spec.js:55	API Testing Module > PUT Update Post API	825ms
api/api.spec.js:73	API Testing Module > DELETE Post API	351ms
api/api.spec.js:92	API Testing Module > Validate Response Title	128ms
api/api.spec.js:103	API Testing Module > Validate Response Body	132ms
api/api.spec.js:117	API Testing Module > Validate Content Type	120ms
api/api.spec.js:131		

The bottom screenshot shows a VS Code editor with the test code for `support.spec.js`. The code defines a `testData` array and a `test` function that runs 60 tests using Playwright.

```
const testData = [
  // ... (60 test cases) ...
];

test('API Testing Module > GET Posts API', async ({ page }) => {
  // ... (test logic) ...
});
```

Concepts Used:

- APIRequestContext
- Assertions
- JSON Validation

7. TEST EXECUTION SUMMARY

Total Test Executions: 396

Browsers:

- Chromium
- Firefox
- WebKit

Execution Status: 100% Passed

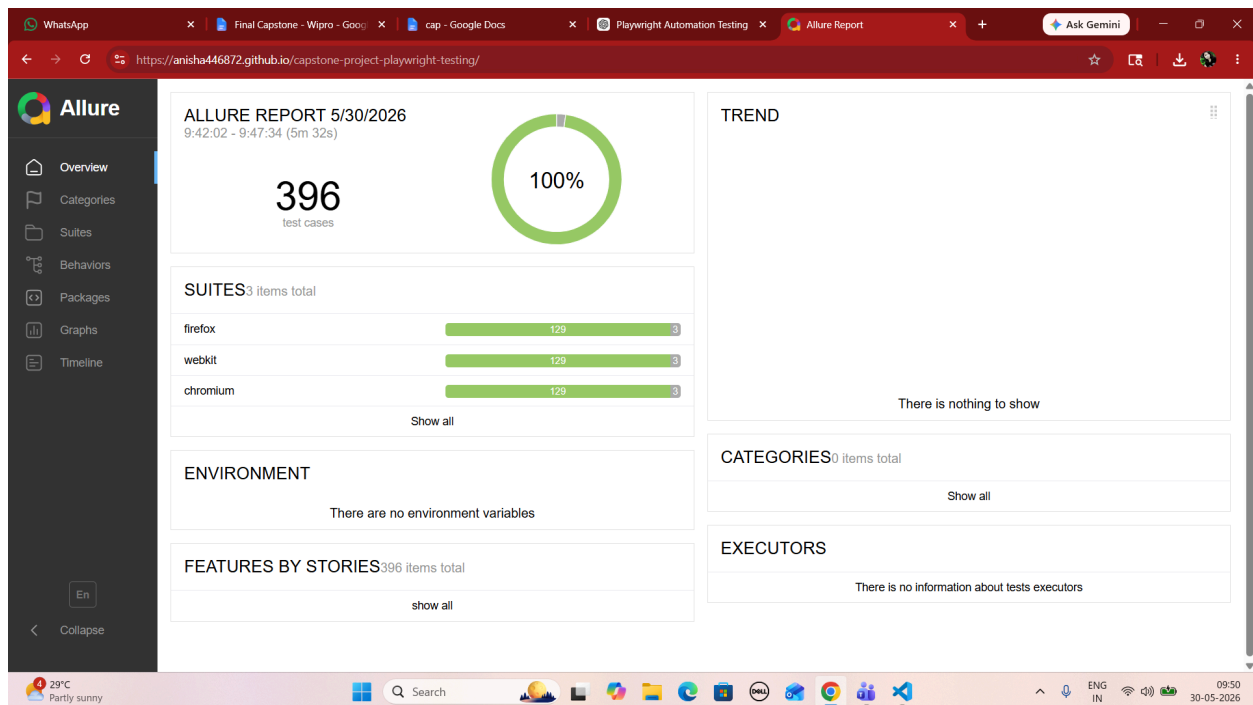


Figure 7.1 – Allure Overview Dashboard

8. CROSS BROWSER TESTING

The framework executes tests across Chromium, Firefox, and WebKit.

Benefits:

- Browser compatibility validation
- Consistent user experience
- Wider test coverage

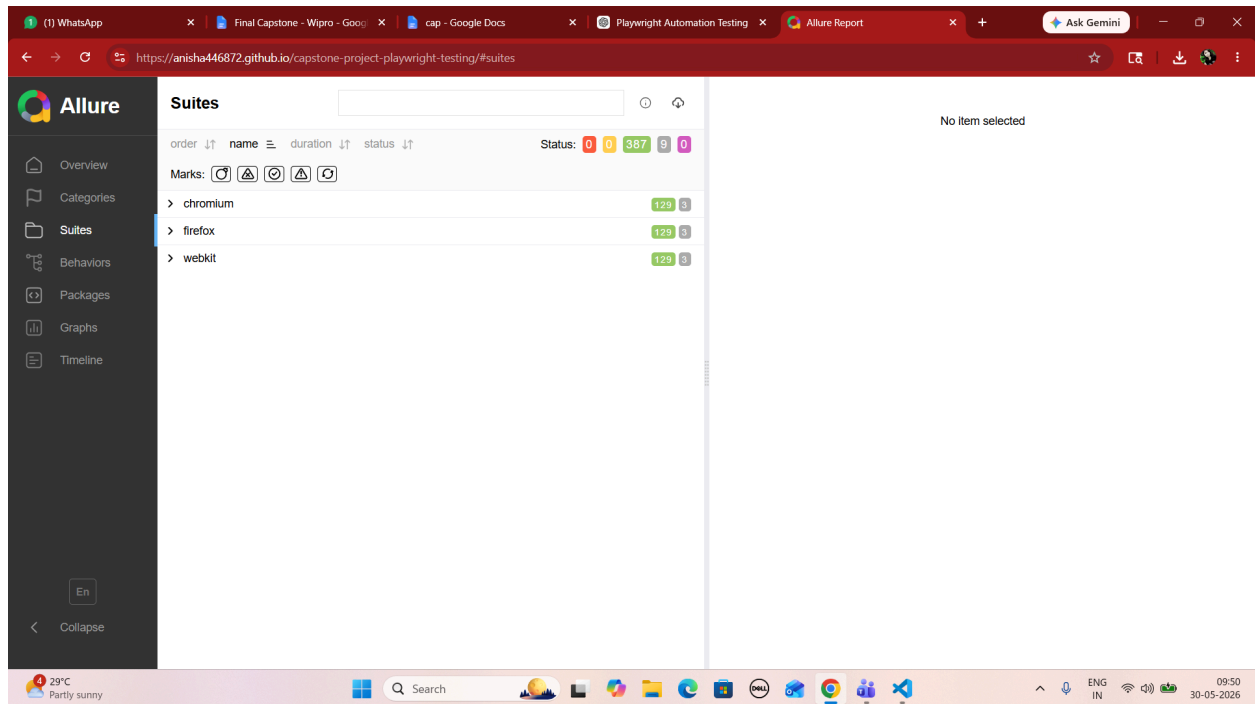


Figure 8.1 – Cross Browser Execution Results

9. API TESTING

The API module validates REST endpoints using Playwright APIRequestContext.

Validations:

- Status Codes
- Headers
- JSON Responses
- Response Time
- Request Methods

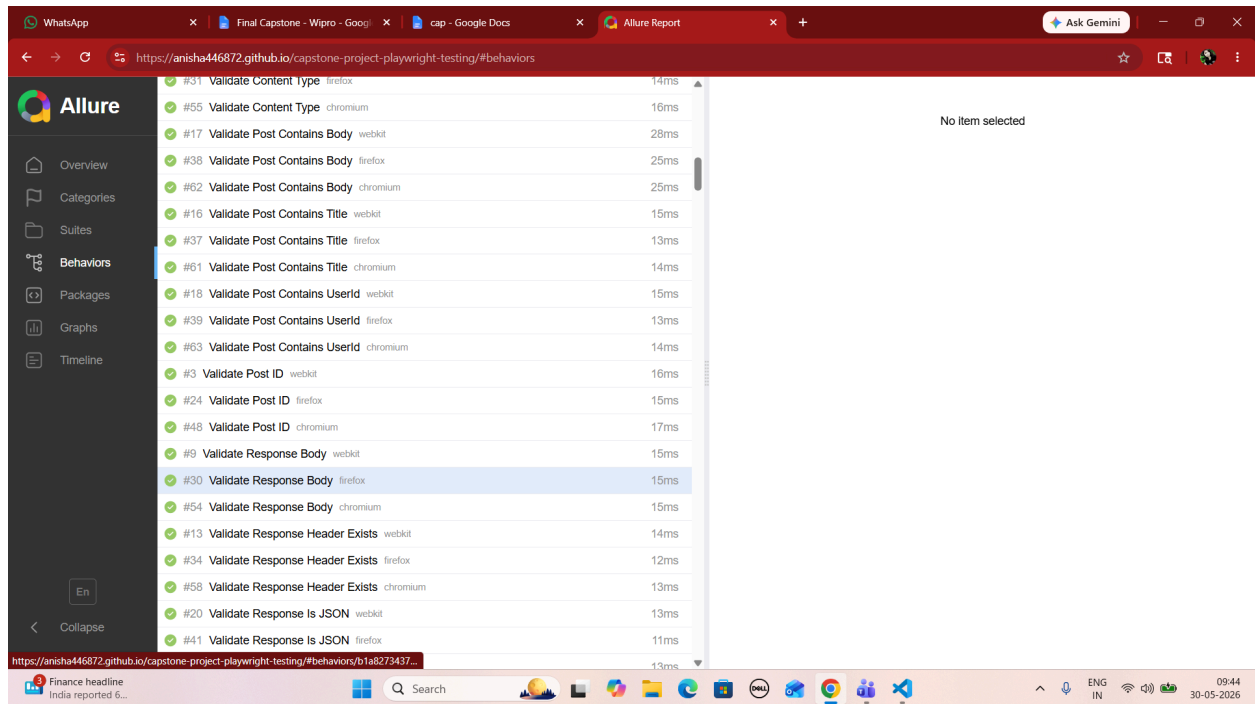


Figure 9.1 – API Validation Test Cases

10. REPORTS AND RESULTS

Allure Features:

- Execution Summary
- Test Case Tracking
- Browser-wise Results
- Test Duration Analysis
- Failure Tracking

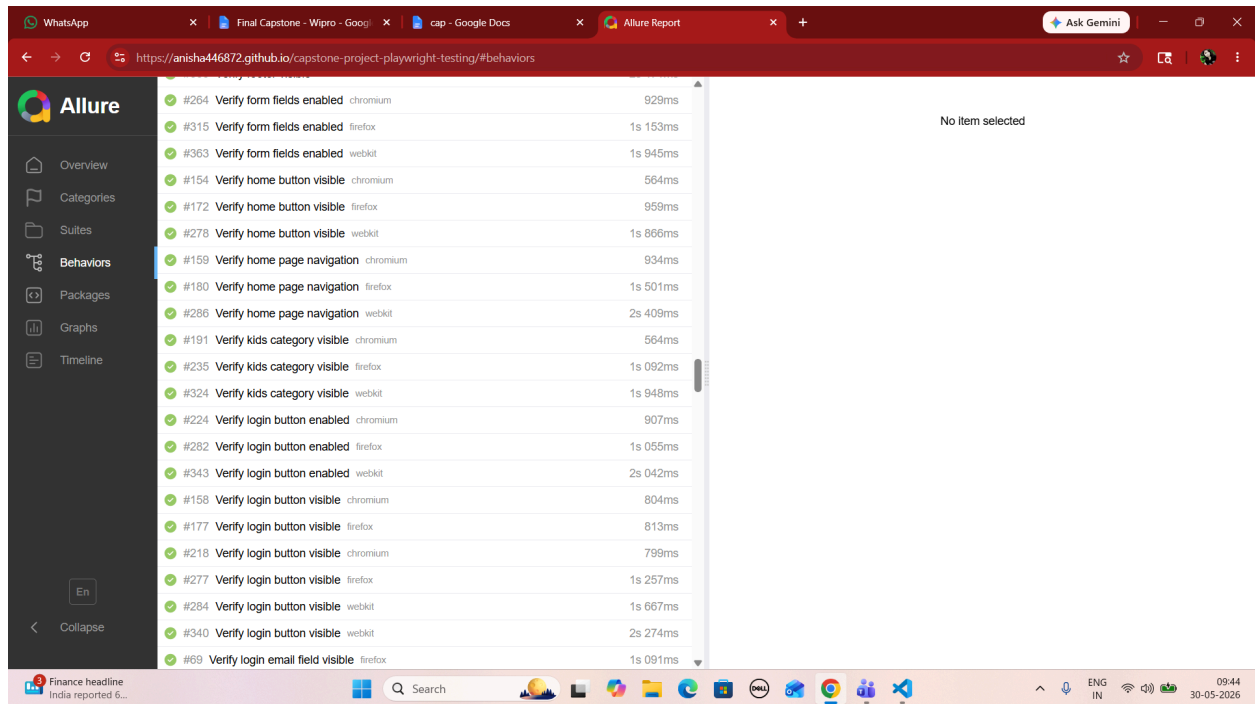


Figure 10.1 – Detailed Test Execution Report

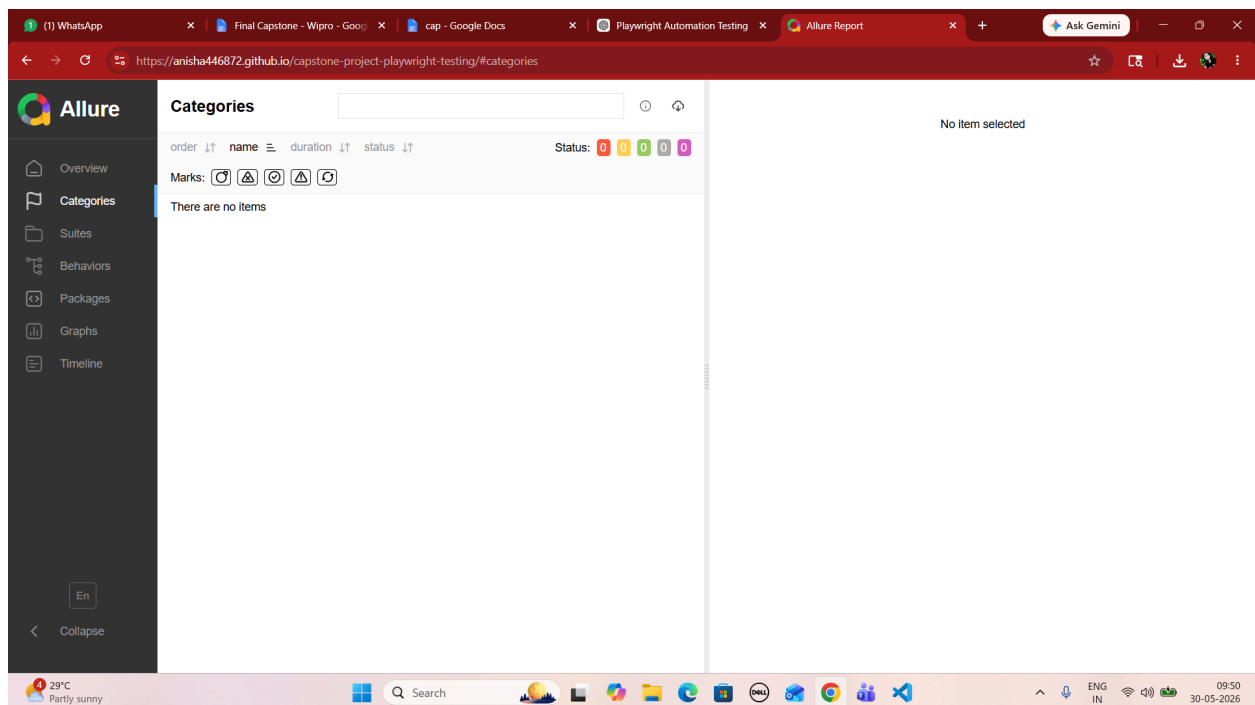


Figure 10.2 – Categories Report

11. GITHUB DEPLOYMENT

Commands Used:

```
git add .
```

```
git commit -m "project update"
```

```
git push origin main
```

Playwright Execution:

```
npx playwright test
```

Allure Report:

```
allure generate allure-results --clean -o allure-report
```

```
allure open allure-report
```

12. CONCLUSION

The project successfully demonstrates automation testing of an E-Commerce application using Playwright Framework.

The framework includes UI Automation, API Automation, Cross Browser Testing, Page Object Model Design, GitHub Actions CI/CD, and Allure Reporting.

A total of 396 test executions were completed successfully with a 100% pass rate, validating the stability and effectiveness of the automation framework.